

# Title

Name<sup>1</sup>, <author name>, Denis Cornaz<sup>1</sup>, Fabio Furini<sup>1</sup>, Enrico Malaguti<sup>1</sup>

<sup>1</sup>Address

<sup>2</sup><address>

<sup>3</sup><address>

<sup>4</sup>LAMSADE, Université Paris-Dauphine, Place du Maréchal de Lattre de Tassigny, 75775 Paris, France  
denis.cornaz@dauphine.fr

<sup>5</sup>LAMSADE, Université Paris-Dauphine, Place du Maréchal de Lattre de Tassigny, 75775 Paris, France  
fabio.furini@dauphine.fr

<sup>6</sup>DEI, University of Bologna, Viale Risorgimento 2, 40136 Bologna, Italy  
enrico.malaguti@unibo.it

## Abstract

In Vertex Coloring Problems, one is required to assign a color to each vertex of an undirected graph in such a way that adjacent vertices receive different colors, and the objective is to minimize the cost of the used colors. In this work we solve four different coloring problems formulated as Maximum Weight Stable Set Problems on an associated graph. We exploit the transformation proposed by Cornaz and Jost [1], where given a graph  $G$ , an auxiliary graph  $\hat{G}$  is constructed, such that the family of all stable sets of  $\hat{G}$  is in one-to-one correspondence with the family of all feasible colorings of  $G$ . The transformation in [1] was originally proposed for the classical Vertex Coloring and the Max-Coloring problems; we extend it to the Equitable Coloring Problem and the Bin Packing Problem with Conflicts. [We discuss the relation between the Maximum Weight Stable formulation and a polynomial-size formulation for the VCP, proposed by Campe^lo, Corre^a and Campos \[4\] and called the \*Representative formulation\*.](#) We report extensive computational experiments on benchmark instances of the four problems, and compare the solution method with the state-of-the-art algorithms. By exploiting the proposed method, we largely outperform the state-of-the-art algorithm for the Max-coloring Problem, and we are able to solve, for the first time to proven optimality, 14 Max-coloring and 2 Equitable Coloring instances.

**Keywords:** Vertex Coloring , Stable Set , Mixed Integer Linear Programming , Computational Experiments.

## 1 Introduction

In Vertex Coloring Problems, one is required to assign a color to each vertex of an undirected graph in such a way that adjacent vertices receive different colors, and the objective is to

minimize the cost of the used colors. In the classical Vertex Coloring Problem (VCP), all colors have the same cost, hence, the objective is to minimize the number of used colors. The Equitable Coloring Problem, is a VCP with the additional restriction that subsets of vertices receiving the same color, denoted as *color classes*, differ in cardinality of at most one unit. The Max-coloring Problem is defined as a VCP where each vertex has a positive weight, and the cost of a color is given by the maximum weight of the vertices in the corresponding color class. Finally, by considering a VCP with a positive weight associated with each vertex, and imposing that the total weight of the vertices in a color class does not to exceed a given capacity, we obtain a VCP with capacity constraints. Because the problem generalizes the Bin Packing Problem as well, it is known in the literature as Bin Packing Problem with Conflicts. All mentioned problems are NP-hard.

Coloring problems are very challenging from the computational viewpoint and have several relevant applications, including, just to mention a few, scheduling [2], timetabling [3], frequency assignment [4], register allocation [5] and communication networks [6] (see Malaguti and Toth, [7]).

**Problems definition** Let  $G$  be a graph with vertex set  $V(G)$  and edge set  $E(G)$ . A stable set  $S \subseteq V(G)$  is a subset of vertices containing no edge, a clique  $K \subseteq V(G)$  is a subset of vertices inducing a complete subgraph. The stable sets of  $G$  are the cliques of the complementary graph  $\overline{G}$  of  $G$ .

Given a weight  $c \in \mathbb{Z}^{V(G)}$ , the *Max Weight Stable Set Problem (MWSSP)* is to determine a stable set  $S$  of  $G$  maximizing  $\sum_{v \in S} c(v)$ . We denote by  $\alpha(G, c)$  the optimum of MWSSP. The MWSSP can be naturally formulated as a (Mixed-)Integer Linear Program (MIP) since  $\alpha(G, c) = \max c^\top x$  over  $x$  in  $STAB(G)$ , that is, the set of vectors of  $\mathbb{R}^{V(G)}$  satisfying

$$\begin{cases} x_u + x_v \leq 1 & uv \in E(G) \\ x_v \in \{0, 1\} & v \in V(G) \end{cases}$$

A *p-coloring of  $G$*  is a partition  $\mathcal{S} = S_1, \dots, S_p$  of  $V(G)$  into  $p$  stable sets  $S_i$ , where each stable set represents a color. The *Vertex Coloring Problem (VCP)* is to find a *p-coloring of  $G$  with a minimum number of colors  $p$* . We denote by  $\chi(G)$  the optimum of VCP. The VCP is a very challenging problem from the computational viewpoint, for which state-of-the-art exact methods can fail in optimally solving instances with more than 200 vertices. The best performing exact methods for the VCP are based on the Set Covering formulation of the problem, where binary variables are associated with stable sets of  $G$ . Since stable sets of an arbitrary graph  $G$  are in exponential number with respect to the graph size, Set Covering formulations require column generation techniques and Branch-and-Price algorithms for managing the exponentially many variables. The recent algorithms by Malaguti, Monaci and Toth [8], Gualandi and Malucelli [9], and Held, Cook and Sewell [10], are all very sophisticated implementations of a Branch-and-Price algorithm, embedding specialized (meta)heuristic procedures [8], constraint programming techniques [9] and improved algorithms for the column generation subproblem [10]. For random

graphs, competitive experimental results are obtained by implicit enumeration schemes based on the DSATUR algorithm by Brélaz [11], see, e.g., [12]. We also mention the Branch-and-Cut approach by Méndez-Díaz and Zabala [13], which is effective for some special classes of graphs.

Given a graph  $G$ , a coloring  $\mathcal{S} = S_1, \dots, S_p$  is *equitable* if  $|S_i| - |S_j| \leq 1$  for each  $i, j = 1, \dots, p$ . The *Equitable Coloring Problem* (ECP) is to find an equitable coloring with minimum  $p$ . We denote by  $\chi_{\text{eq}}(G)$  the optimum of ECP. The most recent mathematical programming contributions to the exact solution of the ECP include the Branch-and-Cut algorithm by Bahiense et al [14], which exploits an asymmetric formulation; and the MIP formulation by Méndez-Díaz, Nasini and Savarín [15], which is strengthened by valid inequalities derived from a polyhedral study, and can be solved directly by a MIP solver. In addition, a DSATUR based Branch-and-Bound algorithm was recently proposed by Méndez-Díaz, Nasini and Savarín [16]. To the best of our knowledge, no extended formulation with variables associated with stable sets of  $G$  was proposed. A possible reason is the non straightforward definition of equitable cardinality constraints for the color classes in this setting.

Given a graph  $G$  and vertex weights  $c \in \mathbb{Z}^{V(G)}$ , the *Max-coloring Problem* (also known as *Weighted VCP*, see [17, 18]) is the problem of determining a coloring  $\mathcal{S} = S_1, \dots, S_p$  of  $G$  which minimizes  $\psi(\mathcal{S}) := \sum_{i=1}^p c_i$  where  $c_i = \max_{v \in S_i} c(v)$ . We denote by  $\chi_{\text{max}}(G, c)$  the optimum of Max-col. When  $c$  is a unit vector,  $\psi(\mathcal{S}) = p$  and Max-col reduces to the VCP. The best performing exact method for the Max-col is the Branch-and-Price algorithm by Furini and Malaguti [18], which can solve instances with up-to 100 vertices.

Given a graph  $G$ , vertex weights  $c \in \mathbb{Z}^{V(G)}$ , and an nonnegative integer  $\kappa$ , the *Bin Packing Problem with Conflict* (BPPC) is to determine a coloring  $\mathcal{S} = S_1, \dots, S_p$  of  $G$  which minimizes  $p$  so that the weight of the stable sets  $c(S_i) := \sum_{v \in S_i} c(v)$  does not exceed  $\kappa$ . The optimum is denoted by  $\chi_{\text{bp}}(G, c, \kappa)$ . State-of-the-art algorithms for the BPPC, recently proposed by Fernandes-Muritiba et al. [19], Elhedhli et al. [20], Sadykov and Vanderbeck [21], exploit a Set Covering formulation and implement a Branch-and-Price framework. Apparently, the presence of capacity constraints on the cardinality of the color classes reduces the practical difficulty of solving the BPPC. The mentioned algorithms can solve to optimality instances with up to 500 vertices.

In this work we solve the VCP, ECP, Max-col and BPPC by reformulating them as MWSSPs on an associated graph. We exploit the transformation proposed by Cornaz and Jost [1], where given a graph  $G$ , an auxiliary graph  $\hat{G}$  is constructed, such that the family of all stable sets of  $\hat{G}$  is in one-to-one correspondence with the family of all feasible colorings of  $G$ . The transformation was originally proposed for the VCP and for the Max-col. We extend the transformation to the ECP and the BPPC; in these cases additional constraints have to be defined for the MWSSP. The advantage of this approach relies on simplicity: it allows us to solve coloring problems by solving MWSSPs, which can be tackled by problem specific algorithms, or can be formulated as MIPs of polynomial size, and solved by a general purpose MIP solver. We discuss the relation between the MWSSPs reformulation and a polynomial-size formulation for the VCP, proposed by Campêlo, Corrêa and Campos [22] and called the *Representative formulation*. To

the best of our knowledge the computational performance of this latter formulation has not been investigated in the literature. Instead, computational results are presented by Bahiense et al. [14], where a Branch-and-Cut algorithm based on the Representative formulation is developed for the Equitable Coloring problem.

The transformation by Cornaz and Jost [1] was exploited by Bonomo, Giandomenico and Rossi to derive polynomial-time algorithms for special classes of graphs [23]. Recently, the same transformation was exploited by Furini, Gabrel and Ternier [24] to derive strong and fast lower bounds for the VCP, which are then used to design a DSATUR based Branch-and-Bound algorithm. The transformation was also exploited in [25] to improve the Theta-Lovász lower bound for VCP.

Given the transformation we exploit, where  $\hat{G}$  has one vertex for each edge in the complement graph of  $G$ , the method is notably promising for instances where  $G$  is dense, thus producing auxiliary graphs  $\hat{G}$  of manageable size.

## 2 “Representative” or “Sandwich graph” formulation

Let us recall the Representative formulation introduced in [22] for the VCP. Observe that any  $p$ -coloring  $S_1, \dots, S_p$  of any given graph  $G = (V, E)$  is also a partition into  $p$  cliques of its complementary graph  $\bar{G} = (V, \bar{E})$ . Assume that  $\vec{\bar{G}} = (V, \vec{\bar{E}})$  is an orientation of  $\bar{G}$  so that every clique of  $\bar{G}$  is acyclic. For instance,  $\vec{\bar{G}}$  could be an acyclic orientation but it is not strictly necessary, a necessary and sufficient condition is that  $\vec{\bar{G}}$  has no 3-dicycle. (Indeed, if a complete graph has an orientation with dicycles, the minimum length of a dicycle is necessarily three).

So, thanks to the way the orientation was chosen, in every (nonempty) clique  $K \subseteq V$  of  $\vec{\bar{G}}$  there is a unique vertex  $v \in K$  which is not the head of any arc in  $K$ . We call  $v$  the *representative* of  $K$ . Moreover, there is a unique star  $\vec{S} \subseteq \vec{\bar{E}}$ , composed of the  $|K| - 1$  arcs having tail  $v$  and whose head is a vertex in  $K \setminus \{v\}$ . So in  $\vec{\bar{G}}$ , any clique  $K$  has both a unique representative  $v$ , and a unique star  $\vec{S}$  (which may be empty if  $K$  is a 1-clique). A *representative set* of  $G$ , is a subset of vertices  $v_1, \dots, v_p$  so that there is a clique partition  $K_1, \dots, K_p$  in  $\vec{\bar{G}}$ , where  $v_i$  is the representative of  $K_i$ .

Thus, any given  $p$ -coloring  $S_1, \dots, S_p$  of  $G$  is also a clique partition in  $\vec{\bar{G}}$ , which has a unique representative set  $v_1, \dots, v_p$ , and uniquely defined stars  $\vec{S}_1, \dots, \vec{S}_p$  so that any vertex is either a representative or a leaf of some star. This way, finding the chromatic number of  $G$  amounts to find a representative set with minimum cardinality. This last problem can be formulated in the

space  $(x, y) \in \mathbb{R}^{V \times \vec{E}}$  as follows

$$\chi(G) = \min \sum_{v \in V} y_v \quad \text{s.t.} \quad (1)$$

$$y_v + \sum_{e \in \delta^-(v)} x_e = 1 \quad v \in V \quad (2)$$

$$x_{vw} \leq y_v \quad v \in V, vw \in \vec{E} \quad (3)$$

$$x_{vu} + x_{vw} \leq y_v \quad v, u, w \in V : vu, vw \in \vec{E}, uw, wu \notin \vec{E} \quad (4)$$

$$y \in \{0, 1\}^V, x \in \{0, 1\}^{\vec{E}}, \quad (5)$$

where  $\delta^-(v)$  denotes the set of arcs in  $\vec{E}$  with head  $v$ . In the above formulation,  $y$  is the characteristic vector of a representative set  $v_1, \dots, v_p$ , and  $x$  is the characteristic vector of the corresponding stars  $\vec{S}_1, \dots, \vec{S}_p$  of  $\vec{G}$ . Constraints (2) and (3) ensure that each vertex is either a representative or the leaf of some star, and constraints (7) ensure that every star spans a stable set of  $G$  (clique of  $\overline{G}$ ). Notice that we can project out the  $x$  variables using Fourier-Motzkin elimination, and we obtain

$$\chi(G) = |V| - \max \sum_{e \in \vec{E}} x_e \quad \text{s.t.} \quad (6)$$

$$x_{vw} + \sum_{e \in \delta^-(v)} x_e \leq 1 \quad v \in V, vw \in \vec{E} \quad (7)$$

$$x_{vu} + x_{vw} + \sum_{e \in \delta^-(v)} x_e \leq 1 \quad v, u, w \in V : vu, vw \in \vec{E}, uw, wu \notin \vec{E} \quad (8)$$

$$x \in \{0, 1\}^{\vec{E}}. \quad (9)$$

One can observe that the feasible vectors  $x$  are in actual fact the characteristic vectors of the stable sets of an auxiliary graph  $\hat{G}$  of  $\vec{G}$  described by Cornaz and Jost [1]. In order to build  $\hat{G}$ , consider the line-graph  $L(\overline{G})$  of the complementary graph  $\overline{G}$  of  $G$ , that is,  $L(\overline{G})$  has vertex-set the edge-set of  $\overline{G}$  and two vertices are linked in  $L(\overline{G})$  if the two corresponding edges are adjacent in  $\overline{G}$ . The auxiliary graph  $\hat{G}$  is obtained from  $L(\overline{G})$  by removing each edge of  $L(\overline{G})$  corresponding to a simplicial pair of arcs of  $\vec{G}$ , that is, two arcs with the same tail and with their heads linked by another arc.

In other words, the vectors  $x$  satisfying (7)-(9) are exactly the vector  $x$  satisfying (11)-(14) and

the following MIP is a formulation for the VCP:

$$\max \sum_{e \in \vec{E}} x_e \quad \text{s.t.} \quad (10)$$

$$x_{vu} + x_{vw} \leq 1 \quad v, u, w \in V : vu, vw \in \vec{E}, uw, wu \notin \vec{E} \quad (11)$$

$$x_e + x_f \leq 1 \quad v \in V, e \in \delta^-(v), f \in \delta^+(v) \quad (12)$$

$$x_e + x_f \leq 1 \quad v \in V, e, f \in \delta^-(v) \quad (13)$$

$$x \in \{0, 1\}^{\vec{E}} \quad (14)$$

where we denote by  $\delta^+(v)$  the set of arcs in  $\vec{E}$  with tail  $v$ . Constraints (11),(12),(13) and (14) define  $\text{STAB}(\hat{G})$ , where  $\hat{G}$  is the auxiliary graph defined above. For Max-col, weights  $\hat{c}_e$  for arcs  $e \in \vec{E}$  are introduced and the problem becomes a MWSSP.

In [1] it is also explain how to transform a weighted graph  $(G, c)$  to an auxiliary weighted graph  $(\hat{G}, \hat{c})$  so that:

**Theorem 1 ([1]).**  $\alpha(\hat{G}, \hat{c}) + \chi_{\max}(G, c) = \mathbf{1}^\top c.$

The first step is to order the vertices  $v_1, \dots, v_n$  of  $V$  so that  $i < j$  if and only if  $c(v_i) \geq c(v_j)$ . (The order is uniquely defined if and only if all the weights are different). Now we let  $\vec{G}$  be the corresponding acyclic orientation of  $\overline{G}$ , that is, each edge  $uv$  of  $\overline{G}$  is orientated from  $u$  to  $v$  whenever  $u = v_i$  and  $v = v_j$  with  $i < j$ . (Hence,  $\vec{G}$  is not necessarily uniquely defined if some vertices have the same weight). Each edge of  $L(\overline{G})$  corresponds to a pair of incident arcs of  $\vec{G}$ . The weight  $\hat{c}_e$  of a vertex  $e$  of  $\hat{G}$  (arc of  $\vec{G}$ ) is  $c_v$  where  $v$  is the head of  $e = uv$  (i.e., the vertex of minimum weight).

A *simplicial star* of  $\vec{G}$  is a subset of arcs all having the same tail and whose heads are pairwise linked. For any coloring  $S_1, \dots, S_p$  of  $G$ , each  $S_i$  is a clique of  $\overline{G}$ , and, since the orientation of  $\overline{G}$  is acyclic, any of these cliques  $S_i$  has a unique representative  $v_i$ , which is the center of a simplicial star  $\vec{S}_i$  of  $\vec{G}$  which spans  $S_i$ . Remark that

$$|S_i| - |\vec{S}_i| = 1 \quad (15)$$

Moreover,

$$\sum_{v \in S_i} c_v - \sum_{e \in \vec{S}_i} \hat{c}_e = c_{v_i} \quad \text{and} \quad c_{v_i} = \max_{v \in S_i} c_v \quad (16)$$

A *simplicial stellar forest* of  $\vec{G}$  is a subset of arcs of  $\vec{G}$  which can be partitioned into vertex-disjoint simplicial stars. Observe that given a simplicial stellar forest  $\vec{F} = \vec{S}_1 \cup \dots \cup \vec{S}_p$  of  $\vec{G}$ , this corresponds to a stable set in  $\hat{G}$ . Since the converse holds as well, it follows that there is a 1-to-1 correspondence between the stable sets of  $\hat{G}$  and the clique partition of  $\overline{G}$ , and hence with the colorings of  $G$ . In the following, we let  $v_1, \dots, v_p$  be the centers of the simplicial stars

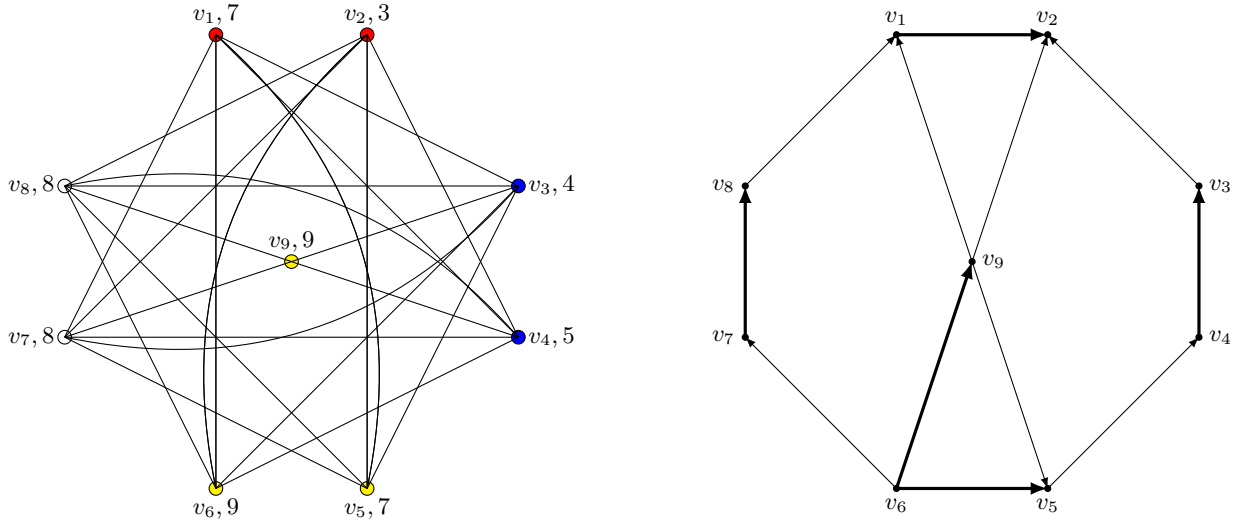
of  $\vec{G}$  (it is a representative set), and we will use the following

$$\text{If } v \in \{v_1, \dots, v_p\}, \text{ then } \delta^-(v) \cap \vec{F} = \emptyset, \text{ otherwise } |\delta^-(v) \cap \vec{F}| = 1 \quad (17)$$

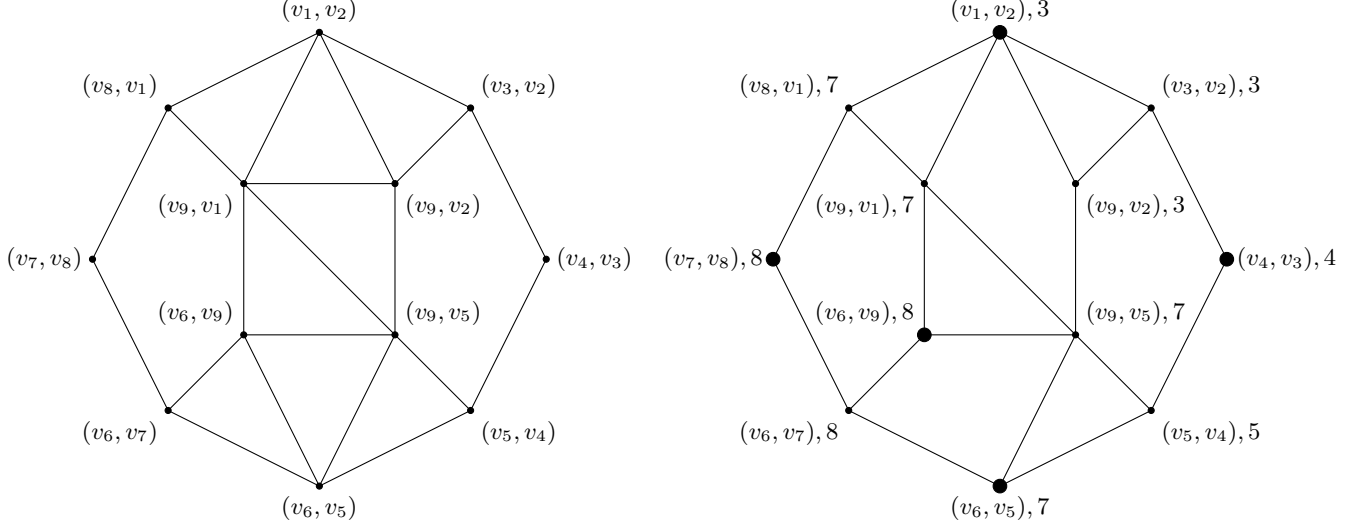
where  $\delta^-(v)$  is the subset of  $\vec{E}$  the head of which is  $v$ , and an isolated vertex is the center of an empty star. Similarly,  $\delta^+(v)$  is the set of all arcs of  $\vec{G}$  the tail of which is  $v$ .

Theorem 1 follows from (16) by summing over all centers  $v_i$ . It allows to formulate any instance of Max-col as an instance of MWSSP.

In Figures 1 and 2, we depict an example of the transformation. On the left of Figure 1 we report a graph  $G$  with 9 vertices and 24 edges. For each vertex  $v$  we report the weight  $c_v$ . On the right of the figure we report the oriented complement graph  $\vec{G}$ . The simplicial stellar forest  $\vec{F}$  is depicted by bold arrows. On the left of Figure 2, we report the line graph  $L(\vec{G})$ , where each vertex corresponds to a pair of incident arcs of  $\vec{G}$  (not depending on the orientation). On the right of the figure, we depict the auxiliary graph  $\hat{G}$ , where edges corresponding to simplicial pairs in  $\vec{G}$  (i.e.,  $(v_9, v_1)(v_9, v_2)$  and  $(v_6, v_9)(v_6, v_5)$ ) are removed. For each vertex  $u, v$  we report the corresponding weight  $c_{u,v} = c_v$ . Vertices in the stable set corresponding to  $\vec{F}$  are represented by larger circles.



**Figure 1:** Example: a graph  $G$  (left) and the orientation of its complement  $\vec{G}$  (right).



**Figure 2:** Example: the line-graph  $L(\overline{G})$  (left) and auxiliary graph  $\hat{G}$  (right).

In the two following propositions, we give MIP formulations for ECP and BPPC. Both are obtained from the basic MWSSP formulation in  $\hat{G}$  by adding constraints.

**Proposition 1.**  $\chi_{\text{eq}}(G) = \min \mathbf{1}^\top x$  over all  $x \in \text{STAB}(\hat{G})$  satisfying

$$x(\delta^+(u)) - x(\delta^+(v)) \leq 1 + x(\delta^-(v))(|\delta^+(u)| - 1) \quad \text{for all } u, v \in V \quad (18)$$

where  $x(A)$  stands for  $\sum_{a \in A} x_a$ .

*Proof of Proposition 1.* If  $u$  is not a center, then  $x(\delta^+(v)) = 0$  and the inequality is trivially satisfied. If  $v$  is not a center, then by (17), we have  $x(\delta^-(v)) = 1$  and again the inequality is trivially satisfied. When both  $u$  and  $v$  are centers, the constraints ensures that  $|\vec{S}_u| - |\vec{S}_v| \leq 1$  where  $\vec{S}_u, \vec{S}_v$  are the simplicial stars of  $u, v$ . By (15), the coloring is 1-balanced. (*end of proof*)

**Proposition 2.**  $\chi_{\text{bp}}(G, c, \kappa) = \min \mathbf{1}^\top x$  over all  $x \in \text{STAB}(\hat{G})$  satisfying

$$c_v + \sum_{w: vw \in \delta^+(v)} c_w x_{vw} \leq \kappa \quad \text{for all vertex } v \in V \quad (19)$$

*Proof of Proposition 2.* If  $v$  is not a center, then  $c_v \leq \kappa$  otherwise the problem is unfeasible. If  $v$  is a center of a simplicial star  $\vec{S}_v$ , then the left-hand-side of the constraint is equal to the weight  $c(S_v)$  of the corresponding stable set. (*end of proof*)

### 3 Computational Experiments

We considered VCP, Max-col, ECP and BPPC instances, we derived the auxiliary graph  $\hat{G} = (\hat{V}, \hat{E})$  as described in the previous section, and solved the resulting MSSP formulated as the MIP (10)-(14). For Max-col, weights  $\hat{c}_e$  for arcs  $e \in \vec{E}$  are considered. Additional constraints (18) and (19) are considered in the case of the ECP and BPPC, respectively (see Section 2). In the following, we denote this solution technique by  $MWSS(\hat{G})$ .

In addition, we tested the Representative formulation (1)-(5), we denote this solution technique by  $REPf(G)$ . Both MIP formulations are solved by the general purpose commercial MIP solver CPLEX 12.6 (see also Section 3.1).

Results are compared with state-of-the-art algorithms for the corresponding problems, all based on Branch-and-Price for the VCP, Max-col and BPPC, and Branch-and-Cut algorithms and a Branch-and-Bound algorithm for the ECP. An important parameter in these experiments is the *density* of the graph, denoted by  $\delta$ , which is the number of edges divided by the maximum possible number of edges (so  $\delta(G) + \delta(\bar{G}) = 1$ ).

#### 3.1 Algorithms for the Maximum Weight Stable Set Problem.

MWSSPs can be solved by means of specialized algorithms, or formulated as a MIP (see Section 1) and directly tackled by a MIP solver.

In our preliminary experiments we considered the following algorithms for solving MWSSPs: i) the Cliquer algorithm by Ostergard, [26]; ii) the Combinatorial Branch and Bound algorithm by Held et al. [10]; iii) the MIP formulation of the MWSSP solved by CPLEX 12.6.

According to our experiments, the best choice is to tackle directly the MIP formulation of the MWSSP by CPLEX. This result is partially in contrast to what could be expected, since specialized algorithms are known to be faster when a MWSSP has to be solved as a column generation subproblem in a Branch-and-Price algorithm (see, e.g., [8, 10, 18]). Note that in the column generation case, the MWSSP to be solved are defined on smaller-size graphs. From our computational experience, for very large graphs, like the ones obtained from the transformation, specialized algorithms tend to be slower. For example, for the 53 VCP instances considered in the next section, within a time limit of 1h of computing time, CPLEX was able to solve to optimality the MWSSP on graph  $\hat{G}$  for 38 instances, versus the 3 and 6 instances solved to optimality by Cliquer [26] and the Branch-and-Bound algorithm in [10], respectively. In the following we report results obtained by solving the MIP formulation of the MWSSP (with additional constraints (18) and (18) in the case of the ECP and BPPC, respectively) by CPLEX 12.6.

#### 3.2 Effect of the orientation of the complement graph $\bar{G}$

The orientation  $\vec{E}$  of the complement graph  $\bar{G}$  has a strong effect on the computing time of the  $MWSS(\hat{G})$  and  $REPf(G)$  solution procedures, when the VCP, ECP and BPPC are considered. We observed that, compared with a random acyclic orientation, improved results are obtained

when arcs in  $\overrightarrow{E}$  are oriented in such a way that the degree (number of adjacent vertices) of the tail is not smaller than the degree of the head. Indeed, we experimentally observed that this orientation produces an auxiliary graph  $\hat{G}$  containing larger cliques, and hence the  $MWSS(\hat{G})$  has to solve a stable set problem which tends to be less challenging for a MIP solver. Similarly, in the  $REPF(G)$ , if the first  $k$  vertices of the ordering define a clique, it is straightforward to show that the linear programming relaxation of model (1)–(5) has at least value  $k$ . Ordering the vertices by non-increasing degree has the effect of producing a MIP with a stronger linear programming relaxation. In the case of the VCP, the overall computing time on the common subset of solved instances is largely reduced (approximately 80%) for both the  $MWSS(\hat{G})$  and the  $REPF(G)$ ; the  $MWSS(\hat{G})$  can solve 3 and  $REPF(G)$  can solve 4 more instances to optimality within time limit, respectively. In the next sections we report results obtained according to this ordering. In the case of the Max-col, the orientation is defined by the relative weight of graph  $G$  vertices.

### 3.3 Computational results for the VCP.

None of the Branch-and-Price algorithms [8], [9] and [10] clearly dominates the others, hence, we compare our results with [8], which reports an extensive set of computational results for benchmark instances. Among the 115 DIMACS benchmark instances<sup>1</sup> considered in [8], we restricted the set to 58 instances for which the auxiliary graph  $\hat{G}$  has at most 30,000 vertices. In Table 1 we report the cardinality of the vertex sets  $|V(G)|$  and  $|V(\hat{G})|$ , and of the edge sets  $|E(G)|$  and  $|E(\hat{G})|$ , respectively, for the original and transformed graphs. Clearly, the size of  $V(\hat{G})$  remains limited for dense graphs  $G$ , which are the instances where the method we study is more promising. To allow a fair comparison on results obtained with different machines, a benchmark program (dfmax) together with a benchmark instance (r500.5) are available. Computing times obtained on different machines can be scaled with respect to the performance obtained on this program. In our experiments, we scale time limits according to the relative speed of the computer we use, but we report original computing times from the respective papers.

In the table we then report the results obtained by the  $MWSS(\hat{G})$  and  $REPF(G)$  with a limit of 6.5 hours of computing time, on a single node of a cluster containing machines equipped with an Intel Xeon E3-1220 processor at 3.10 GHz and 8 GB RAM. This machine needs 4.5 seconds to run the benchmark program on the instance. We report, for each considered instance and each method, a lower bound  $LB$  and an upper bound  $UB$  on the value  $\chi(G)$ , and the corresponding computing time ( $tl$  when time limit is reached), which, for  $MWSS(\hat{G})$  includes the (very small) time for applying the transformation of  $G$  to  $\hat{G}$ . In the next columns, we report the results of the Branch-and-Price algorithm in [8] (columns BP [8]), obtained within 10 hours of computing time on a Pentium 4 PC with 2 GB RAM, which needs 7 seconds to solve the benchmark program on the instance.

On the considered 58 instances, the  $REPF(G)$  method performs very well solving to optimality 41 instances. The  $MWSS(\hat{G})$  method can solve 39 instances, with comparable computing times

---

<sup>1</sup><http://dimacs.rutgers.edu/pub/challenge/graph/>

with some notable exception, for instance DSJC125.1 and instances of the Insertion and FullIns sets, where the computing times of the  $REPf(G)$  are shorter for solved problems. Despite one would expect CPLEX MIP solver to be more effective in solving models formulated as (unweighted) stable set problems, our computational tests showed that instead this is not the case. In particular, we observed that the LP solver of CPLEX tends to spend more time when solving the linear programming relaxation of the  $MWSS(\hat{G})$  than the  $REPf(G)$ . Both methods perform very well when compared with the 39 instances optimally solved by [8]. In particular  $MWSS(\hat{G})$  can solve in less than one hour instance DSJC250.9, which was only recently solved to optimality by [10] with almost 3 hours of computing time.

### 3.4 Computational results for the Max-col.

The best performing algorithm for Max-col, i.e., the Branch-and-Price algorithm [18], can solve instances with up to 100 vertices. This makes the  $MWSS(\hat{G})$  method an excellent candidate alternative for solving the Max-col, since we expect the weighted auxiliary graph  $\hat{G}$  being not too large, and we do not have to spoil the MWSSP formulation with additional constraints. We considered all the instances in [18]. This includes 46 instances from the COLOR02/03/04 benchmark<sup>2</sup> and two sets of instances from matrix-decomposition problems. In Tables 2, 4 and 5 we report the size of the original and transformed graphs, and the results obtained by the presented method (columns  $MWSS(\hat{G})$ ), followed by the results from [18], obtained on the same computer (an i5 linux PC with 8 GB RAM). Time limits are set to 1 hour of computing time for the first to two sets (DIMACS and  $p$ ) and to 4 hours for the last set ( $R$ ), as in [18]. We also tested the  $REPf(G)$  but, for the Max-col, the  $MWSS(\hat{G})$  has a slightly better performance, being able to solve one more instance to proven optimality, with similar computing times on the other solved instances. In contrast with what we have experienced for the VCP, CPLEX better solves the weighted stable set formulation ( $MWSS(\hat{G})$ ) than the weighted extension of the  $REPf(G)$  to the Max-col.

Clearly the results obtained by the  $MWSS(\hat{G})$  method largely outperform those in [18] (obtained on the same computer): concerning the 46 COLOR02/03/04 benchmark, 38 instances are solved to optimality, i.e., 2 more than [18]. By allowing up to 10 hours of computing time (see Table 3), the  $MWSS(\hat{G})$  method can additionally solve to optimality instances R100\_1g and R100\_1gb, and for unsolved instances, the optimality gaps are further reduced and always smaller than those obtained by [18] (while the Branch-and-Price algorithm in [18] does not benefit from additional computing time). Concerning the  $p$  instances, which are all solved to optimality by both methods, the average computing time is reduced of one order of magnitude, from 6.83 to 0.35 seconds. Finally, for the  $R$  instances, [18] reports only 19 optimal solutions, while all the 30 instances are optimally solved by the  $MWSS(\hat{G})$  method.

### 3.5 Computational results for the ECP

For solving the ECP, we consider the MWSSP on graph  $\hat{G}$  and the additional balance constraints (18). Constraints (18) are  $O(n^2)$  in total, since one constraint is defined for each pair of vertices

---

<sup>2</sup><http://mat.gsia.cmu.edu/COLOR02>

| <i>instance</i> | <i>G</i> |          | $\hat{G}$      |                | <i>MWSS</i> ( $\hat{G}$ ) |           |             | <i>REPF</i> ( <i>G</i> ) |           |             | BP [8]    |           |
|-----------------|----------|----------|----------------|----------------|---------------------------|-----------|-------------|--------------------------|-----------|-------------|-----------|-----------|
|                 | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | <i>LB</i>                 | <i>UB</i> | <i>time</i> | <i>LB</i>                | <i>UB</i> | <i>time</i> | <i>LB</i> | <i>UB</i> |
| DSJC125.1       | 125      | 736      | 7,014          | 545,250        | 5                         | 5         | 17314.7     | 5                        | 5         | 65.5        | 5         | 5         |
| DSJC125.5       | 125      | 3,891    | 3,859          | 197,166        | 14                        | 20        | tl          | 14                       | 20        | tl          | 17        | 17        |
| DSJC125.9       | 125      | 6,961    | 789            | 9,503          | 44                        | 44        | 0.5         | 44                       | 44        | 0.5         | 44        | 44        |
| DSJC250.1       | 250      | 3,218    | 27,907         | 4,350,878      | 5                         | 17        | tl          | 5                        | 10        | tl          | 6         | 8         |
| DSJC250.5       | 250      | 15,668   | 15,457         | 1,588,249      | 17                        | 48        | tl          | 18                       | 40        | tl          | 20        | 28        |
| DSJC250.9       | 250      | 27,897   | 3,228          | 79,902         | 72                        | 72        | 1935.2      | 72                       | 72        | 2401.5      | 71        | 72        |
| DSJC500.9       | 500      | 112,437  | 12,313         | 584,336        | 123                       | 132       | tl          | 123                      | 133       | tl          | 123       | 127       |
| DSJR500.1c      | 500      | 121,275  | 3,475          | 39,563         | 85                        | 85        | 1.3         | 85                       | 85        | 1.3         | 85        | 85        |
| queen5_5        | 25       | 160      | 140            | 1,240          | 5                         | 5         | 0           | 5                        | 5         | 0           | 5         | 5         |
| queen6_6        | 36       | 290      | 340            | 5,092          | 7                         | 7         | 0.5         | 7                        | 7         | 0.5         | 7         | 7         |
| queen7_7        | 49       | 476      | 700            | 15,736         | 7                         | 7         | 0.3         | 7                        | 7         | 0.2         | 7         | 7         |
| queen8_8        | 64       | 728      | 1,288          | 40,344         | 9                         | 9         | 150.6       | 9                        | 9         | 34.9        | 9         | 9         |
| queen9_9        | 81       | 1,056    | 2,184          | 90,672         | 9                         | 10        | tl          | 9                        | 10        | tl          | 10        | 10        |
| queen10_10      | 100      | 1,470    | 3,480          | 184,600        | 11                        | 12        | tl          | 10                       | 12        | tl          | 11        | 11        |
| queen11_11      | 121      | 1,980    | 5,280          | 348,040        | 11                        | 13        | tl          | 11                       | 13        | tl          | 12        | 12        |
| queen12_12      | 144      | 2,596    | 7,700          | 616,940        | 12                        | 14        | tl          | 12                       | 15        | tl          | 12        | 13        |
| queen13_13      | 169      | 3,328    | 10,868         | 1,039,688      | 13                        | 16        | tl          | 13                       | 16        | tl          | 13        | 14        |
| queen14_14      | 196      | 4,186    | 14,924         | 1,679,580      | 14                        | 17        | tl          | 14                       | 18        | tl          | 14        | 15        |
| queen15_15      | 225      | 5,180    | 20,020         | 2,617,720      | 16                        | 27        | tl          | 15                       | 19        | tl          | 15        | 15        |
| queen16_16      | 256      | 6,320    | 26,320         | 3,955,952      |                           | 256       | tl          | 16                       | 24        | tl          | 16        | 17        |
| myciel3_        | 11       | 20       | 35             | 150            | 4                         | 4         | 0           | 4                        | 4         | 0           | 4         | 4         |
| myciel4         | 23       | 71       | 182            | 2,051          | 5                         | 5         | 0.2         | 5                        | 5         | 0.2         | 5         | 5         |
| myciel5         | 47       | 236      | 845            | 21,810         | 6                         | 6         | 150.6       | 6                        | 6         | 61.2        | 4         | 6         |
| myciel6         | 95       | 755      | 3,710          | 206,615        | 6                         | 7         | tl          | 6                        | 7         | tl          | 4         | 7         |
| myciel7         | 191      | 2,360    | 15,785         | 1,840,230      | 4                         | 8         | tl          | 4                        | 8         | tl          | 5         | 8         |
| multsol.i.1     | 197      | 3,925    | 15,381         | 1,805,912      | 49                        | 49        | 6.8         | 49                       | 49        | 2           | 49        | 49        |
| multsol.i.2     | 188      | 3,885    | 13,693         | 1,498,197      | 31                        | 31        | 6.2         | 31                       | 31        | 12.3        | 31        | 31        |
| multsol.i.3     | 184      | 3,916    | 12,920         | 1,371,624      | 31                        | 31        | 4.9         | 31                       | 31        | 9.8         | 31        | 31        |
| multsol.i.4     | 185      | 3,946    | 13,074         | 1,396,293      | 31                        | 31        | 5.8         | 31                       | 31        | 10.4        | 31        | 31        |
| multsol.i.5     | 186      | 3,973    | 13,232         | 1,421,732      | 31                        | 31        | 5.4         | 31                       | 31        | 11.1        | 31        | 31        |
| zeroin.i.1      | 211      | 4,100    | 18,055         | 2,303,047      | 49                        | 49        | 9.1         | 49                       | 49        | 10          | 49        | 49        |
| zeroin.i.2      | 211      | 3,541    | 18,614         | 2,383,090      | 30                        | 30        | 11          | 30                       | 30        | 0.5         | 30        | 30        |
| zeroin.i.3      | 206      | 3,540    | 17,575         | 2,185,681      | 30                        | 30        | 11          | 30                       | 30        | 9.4         | 30        | 30        |
| anna            | 138      | 493      | 8,960          | 790,966        | 11                        | 11        | 2.8         | 11                       | 11        | 0.2         | 11        | 11        |
| david           | 87       | 406      | 3,335          | 178,437        | 11                        | 11        | 0.4         | 11                       | 11        | 0.1         | 11        | 11        |
| huck            | 74       | 301      | 2,400          | 108,649        | 11                        | 11        | 0.2         | 11                       | 11        | 0.1         | 11        | 11        |
| jean            | 80       | 254      | 2,906          | 144,975        | 10                        | 10        | 0.4         | 10                       | 10        | 0.1         | 10        | 10        |
| games120        | 120      | 638      | 6,502          | 487,295        | 9                         | 9         | 4.9         | 9                        | 9         | 0.4         | 9         | 9         |
| miles250        | 128      | 387      | 7,741          | 634,554        | 8                         | 8         | 1.8         | 8                        | 8         | 0.2         | 8         | 8         |
| miles500        | 128      | 1,170    | 6,958          | 541,403        | 20                        | 20        | 1.3         | 20                       | 20        | 0.2         | 19        | 20        |
| miles750        | 128      | 2,113    | 6,015          | 436,137        | 31                        | 31        | 0.9         | 31                       | 31        | 2           | 31        | 31        |
| miles1000       | 128      | 3,216    | 4,912          | 323,889        | 42                        | 42        | 0.7         | 42                       | 42        | 3.3         | 42        | 42        |
| miles1500       | 128      | 5,198    | 2,930          | 158,110        | 73                        | 73        | 0.2         | 73                       | 73        | 1           | 73        | 73        |
| 1-Insertions_4  | 67       | 232      | 1,979          | 80,730         | 4                         | 5         | tl          | 5                        | 5         | 15050.7     | 3         | 5         |
| 1-Insertions_5  | 202      | 1,227    | 19,074         | 2,461,400      | 3                         | 6         | tl          | 4                        | 6         | tl          | 3         | 6         |
| 2-Insertions_4  | 149      | 541      | 10,485         | 1,001,021      | 4                         | 5         | tl          | 4                        | 5         | tl          | 3         | 5         |
| 4-Insertions_3  | 79       | 156      | 2,925          | 146,146        | 3                         | 4         | tl          | 4                        | 4         | 13564.5     | 3         | 4         |
| 1-FullIns_4     | 93       | 593      | 3,685          | 205,688        | 5                         | 5         | 17.3        | 5                        | 5         | 4.8         | 4         | 5         |
| 1-FullIns_5     | 282      | 3,247    | 36,374         | 6,487,330      | 5                         | 6         | tl          | 5                        | 6         | tl          | 4         | 6         |
| 2-FullIns_3     | 52       | 201      | 1,125          | 34,190         | 5                         | 5         | 0.3         | 5                        | 5         | 0           | 5         | 5         |
| 2-FullIns_4     | 212      | 1,621    | 20,745         | 2,791,046      | 6                         | 6         | 16021.6     | 6                        | 6         | 115.6       | 5         | 6         |
| 3-FullIns_3     | 80       | 346      | 2,814          | 137,405        | 6                         | 6         | 1.4         | 6                        | 6         | 0.2         | 6         | 6         |
| 4-FullIns_3     | 114      | 541      | 5,900          | 420,462        | 7                         | 7         | 4.3         | 7                        | 7         | 0.4         | 7         | 7         |
| 5-FullIns_3     | 154      | 792      | 10,989         | 1,073,628      | 8                         | 8         | 14.6        | 8                        | 8         | 0.7         | 8         | 8         |
| mug88_1         | 88       | 146      | 3,682          | 206,951        | 4                         | 4         | 6.5         | 4                        | 4         | 0.7         | 4         | 4         |
| mug88_25        | 88       | 146      | 3,682          | 206,953        | 4                         | 4         | 6.9         | 4                        | 4         | 5.3         | 4         | 4         |
| mug100_1        | 100      | 166      | 4,784          | 307,175        | 4                         | 4         | 31          | 4                        | 4         | 5.8         | 4         | 4         |
| mug100_25       | 100      | 166      | 4,784          | 307,175        | 4                         | 4         | 119.6       | 4                        | 4         | 28.6        | 4         | 4         |

**Table 1:** Results for the VCP, DIMACS benchmark instances.

| <i>instance</i> | $G$      |          | $\hat{G}$      |                | $MWSS(\hat{G})$ |      |         | BP [8] |      |        |
|-----------------|----------|----------|----------------|----------------|-----------------|------|---------|--------|------|--------|
|                 | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | $LB$            | $UB$ | $time$  | $LB$   | $UB$ | $time$ |
| DSJC125_1g      | 125      | 736      | 7,014          | 545,250        | 20              | 30   | tl      | 21     | 27   | tl     |
| DSJC125_1gb     | 125      | 736      | 7,014          | 545,250        | 76              | 104  | tl      | 82     | 96   | tl     |
| DSJC125_5g      | 125      | 3,891    | 3,859          | 197,166        | 62              | 82   | tl      | 68     | 82   | tl     |
| DSJC125_5gb     | 125      | 3,891    | 3,859          | 197,166        | 209             | 284  | tl      | 230    | 273  | tl     |
| DSJC125_9g      | 125      | 6,961    | 789            | 9,503          | 169             | 169  | 0.3     | 169    | 169  | 2      |
| DSJC125_9gb     | 125      | 6,961    | 789            | 9,503          | 604             | 604  | 0.4     | 604    | 604  | 5      |
| GEOM100         | 100      | 547      | 4,403          | 271,017        | 65              | 65   | 3.8     | 65     | 65   | 12     |
| GEOM100a        | 100      | 992      | 3,958          | 230,557        | 89              | 89   | 2.4     | 89     | 89   | 2      |
| GEOM100b        | 100      | 1,050    | 3,900          | 225,378        | 32              | 32   | 3.8     | 32     | 32   | 1      |
| GEOM110         | 110      | 638      | 5,357          | 364,224        | 68              | 68   | 59.5    | 68     | 69   | tl     |
| GEOM110a        | 110      | 1,207    | 4,788          | 307,120        | 97              | 97   | 12.9    | 97     | 97   | 7      |
| GEOM110b        | 110      | 1,256    | 4,739          | 302,283        | 37              | 37   | 5.0     | 37     | 37   | 2      |
| GEOM120         | 120      | 773      | 6,367          | 472,507        | 72              | 72   | 12.4    | 72     | 72   | 12     |
| GEOM120a        | 120      | 1,434    | 5,706          | 400,114        | 105             | 105  | 7.0     | 105    | 105  | 11     |
| GEOM120b        | 120      | 1,491    | 5,649          | 393,915        | 35              | 35   | 16.5    | 35     | 35   | 1      |
| GEOM30b         | 30       | 81       | 354            | 5,923          | 12              | 12   | 0.0     | 12     | 12   | 0      |
| GEOM40b         | 40       | 157      | 623            | 14,023         | 16              | 16   | 0.1     | 16     | 16   | 0      |
| GEOM50b         | 50       | 249      | 976            | 27,722         | 18              | 18   | 0.1     | 18     | 18   | 0      |
| GEOM60b         | 60       | 366      | 1,404          | 48,110         | 23              | 23   | 0.5     | 23     | 23   | 0      |
| GEOM70          | 70       | 267      | 2,148          | 91,726         | 47              | 47   | 0.3     | 47     | 47   | 1      |
| GEOM70a         | 70       | 459      | 1,956          | 79,625         | 73              | 73   | 0.4     | 73     | 73   | 1      |
| GEOM70b         | 70       | 488      | 1,927          | 77,712         | 24              | 24   | 0.9     | 24     | 24   | 0      |
| GEOM80          | 80       | 349      | 2,811          | 137,702        | 66              | 66   | 1.1     | 66     | 66   | 2      |
| GEOM80a         | 80       | 612      | 2,548          | 118,686        | 76              | 76   | 1.1     | 76     | 76   | 1      |
| GEOM80b         | 80       | 663      | 2,497          | 114,959        | 27              | 27   | 2.5     | 27     | 27   | 1      |
| GEOM90          | 90       | 441      | 3,564          | 197,010        | 61              | 61   | 2.0     | 61     | 61   | 4      |
| GEOM90a         | 90       | 789      | 3,216          | 168,570        | 73              | 73   | 4.8     | 73     | 73   | 1      |
| GEOM90b         | 90       | 860      | 3,145          | 162,945        | 30              | 30   | 1.7     | 30     | 30   | 1      |
| R100_1g         | 100      | 509      | 4,441          | 273,695        | 19              | 23   | tl      | 20     | 25   | tl     |
| R100_1gb        | 100      | 509      | 4,441          | 273,695        | 76              | 83   | tl      | 76     | 90   | tl     |
| R100_5g         | 100      | 2,456    | 2,494          | 102,327        | 53              | 62   | tl      | 56     | 65   | tl     |
| R100_5gb        | 100      | 2,456    | 2,494          | 102,327        | 203             | 225  | tl      | 212    | 233  | tl     |
| R100_9g         | 100      | 4,438    | 512            | 5,033          | 141             | 141  | 0.1     | 141    | 141  | 0      |
| R100_9gb        | 100      | 4,438    | 512            | 5,033          | 518             | 518  | 0.3     | 517    | 518  | 5      |
| R50_1g          | 50       | 108      | 1,117          | 34,029         | 14              | 14   | 0.8     | 14     | 14   | 1      |
| R50_1gb         | 50       | 108      | 1,117          | 34,029         | 53              | 53   | 1.6     | 53     | 53   | 0      |
| R50_5g          | 50       | 612      | 613            | 12,251         | 37              | 37   | 1.4     | 36     | 37   | 2      |
| R50_5gb         | 50       | 612      | 613            | 12,251         | 135             | 135  | 1.5     | 131    | 135  | 22     |
| R50_9g          | 50       | 1,092    | 133            | 654            | 74              | 74   | 0.0     | 74     | 74   | 0      |
| R50_9gb         | 50       | 1,092    | 133            | 654            | 262             | 262  | 0.0     | 262    | 262  | 0      |
| R75_1g          | 70       | 251      | 2,164          | 92,467         | 18              | 18   | 132.8   | 18     | 18   | 12     |
| R75_1gb         | 70       | 251      | 2,164          | 92,467         | 70              | 70   | 192.2   | 67     | 70   | 447    |
| R75_5g          | 75       | 1,407    | 1,368          | 41,115         | 51              | 51   | 1,056.0 | 49     | 51   | 2,136  |
| R75_5gb         | 75       | 1,407    | 1,368          | 41,115         | 186             | 186  | 989.3   | 179    | 187  | tl     |
| R75_9g          | 75       | 2,513    | 262            | 1,724          | 110             | 110  | 0.0     | 109    | 110  | 0      |
| R75_9gb         | 75       | 2,513    | 262            | 1,724          | 396             | 396  | 0.0     | 396    | 396  | 1      |

**Table 2:** Results for the Max-col, COLOR02/03/04 instances, time limit of 1 hour.

| <i>instance</i> | $G$      |          | $\hat{G}$      |                | $MWSS(\hat{G})$ |      |          |
|-----------------|----------|----------|----------------|----------------|-----------------|------|----------|
|                 | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | $LB$            | $UB$ | $time$   |
| DSJC125_1g      | 125      | 736      | 7,014          | 545,250        | 20              | 25   | tl       |
| DSJC125_1gb     | 125      | 736      | 7,014          | 545,250        | 78              | 95   | tl       |
| DSJC125_5g      | 125      | 3,891    | 3,859          | 197,166        | 63              | 78   | tl       |
| DSJC125_5gb     | 125      | 3,891    | 3,859          | 197,166        | 212             | 263  | tl       |
| R100_1g         | 100      | 509      | 4,441          | 273,695        | 21              | 21   | 28,788.5 |
| R100_1gb        | 100      | 509      | 4,441          | 273,695        | 81              | 81   | 9,362.2  |
| R100_5g         | 100      | 2,456    | 2,494          | 102,327        | 55              | 59   | tl       |
| R100_5gb        | 100      | 2,456    | 2,494          | 102,327        | 210             | 225  | tl       |

**Table 3:** Results for the Max-col, COLOR02/03/04 instances, time limit of 10 hours.

| <i>instance</i> | $G$      |          | $\hat{G}$      |                | $MWSS(\hat{G})$ |       |        | BP [8] |       |        |
|-----------------|----------|----------|----------------|----------------|-----------------|-------|--------|--------|-------|--------|
|                 | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | $LB$            | $UB$  | $time$ | $LB$   | $UB$  | $time$ |
| P06             | 16       | 38       | 82             | 608            | 565             | 565   | 0.0    | 565    | 565   | 0      |
| P07             | 24       | 92       | 184            | 2,112          | 3,771           | 3,771 | 0.0    | 3,771  | 3,771 | 0      |
| p08             | 24       | 92       | 184            | 2,112          | 4,049           | 4,049 | 0.0    | 4,049  | 4,049 | 0      |
| p09             | 25       | 100      | 200            | 2,400          | 3,388           | 3,388 | 0.0    | 3,388  | 3,388 | 0      |
| p10             | 16       | 32       | 88             | 688            | 3,983           | 3,983 | 0.0    | 3,983  | 3,983 | 0      |
| p11             | 18       | 48       | 105            | 904            | 3,380           | 3,380 | 0.0    | 3,380  | 3,380 | 0      |
| p12             | 26       | 90       | 235            | 3,118          | 657             | 657   | 0.0    | 657    | 657   | 0      |
| p13             | 34       | 160      | 401            | 7,048          | 3,220           | 3,220 | 0.0    | 3,220  | 3,220 | 1      |
| p14             | 31       | 110      | 355            | 5,899          | 3,157           | 3,157 | 0.0    | 3,157  | 3,157 | 0      |
| p15             | 34       | 136      | 425            | 7,760          | 341             | 341   | 0.0    | 341    | 341   | 0      |
| p16             | 34       | 134      | 427            | 7,816          | 2,343           | 2,343 | 0.0    | 2,343  | 2,343 | 0      |
| p17             | 37       | 161      | 505            | 10,090         | 3,281           | 3,281 | 0.0    | 3,281  | 3,281 | 0      |
| p18             | 35       | 143      | 452            | 8,523          | 3,228           | 3,228 | 0.0    | 3,228  | 3,228 | 0      |
| p19             | 36       | 156      | 474            | 9,196          | 3,710           | 3,710 | 0.0    | 3,710  | 3,710 | 0      |
| p20             | 37       | 142      | 524            | 10,715         | 1,830           | 1,830 | 0.0    | 1,830  | 1,830 | 1      |
| p21             | 38       | 155      | 548            | 11,471         | 3,660           | 3,660 | 0.0    | 3,660  | 3,660 | 0      |
| p22             | 38       | 154      | 549            | 11,498         | 1,912           | 1,912 | 0.0    | 1,912  | 1,912 | 0      |
| p23             | 44       | 204      | 742            | 18,179         | 3,770           | 3,770 | 0.1    | 3,770  | 3,770 | 0      |
| p24             | 34       | 104      | 457            | 8,723          | 661             | 661   | 0.0    | 661    | 661   | 0      |
| p25             | 36       | 120      | 510            | 10,308         | 504             | 504   | 0.0    | 504    | 504   | 0      |
| p26             | 37       | 131      | 535            | 11,086         | 520             | 520   | 0.0    | 520    | 520   | 0      |
| p27             | 44       | 174      | 772            | 19,356         | 216             | 216   | 0.1    | 216    | 216   | 2      |
| p28             | 44       | 164      | 782            | 19,766         | 1,729           | 1,729 | 0.1    | 1,729  | 1,729 | 0      |
| p29             | 53       | 254      | 1,124          | 34,271         | 3,470           | 3,470 | 0.1    | 3,470  | 3,470 | 0      |
| p30             | 60       | 317      | 1,453          | 50,540         | 4,891           | 4,891 | 0.2    | 4,891  | 4,891 | 1      |
| p31             | 47       | 179      | 902            | 24,580         | 620             | 620   | 0.1    | 620    | 620   | 0      |
| p32             | 51       | 221      | 1,054          | 31,117         | 2,480           | 2,480 | 0.1    | 2,480  | 2,480 | 0      |
| p33             | 56       | 258      | 1,282          | 41,840         | 3,018           | 3,018 | 0.3    | 3,018  | 3,018 | 1      |
| p34             | 74       | 421      | 2,280          | 100,077        | 1,980           | 1,980 | 0.6    | 1,980  | 1,980 | 2      |
| p35             | 86       | 566      | 3,089          | 158,255        | 2,140           | 2,140 | 0.6    | 2,140  | 2,140 | 2      |
| p36             | 101      | 798      | 4,252          | 256,244        | 7,210           | 7,210 | 1.4    | 7,210  | 7,210 | 4      |
| p38             | 87       | 537      | 3,204          | 167,359        | 2,130           | 2,130 | 1.2    | 2,130  | 2,130 | 10     |
| p40             | 86       | 497      | 3,158          | 163,820        | 4,984           | 4,984 | 1.0    | 4,984  | 4,984 | 13     |
| p41             | 116      | 900      | 5,770          | 406,504        | 2,688           | 2,688 | 3.2    | 2,688  | 2,688 | 31     |
| p42             | 138      | 1,186    | 8,267          | 698,948        | 2,466           | 2,466 | 3.2    | 2,466  | 2,466 | 171    |

**Table 4:** Results for the Max-col,  $p$  instances.

| <i>instance</i> | $G$      |          | $\hat{G}$      |                | $MWSS(\hat{G})$ |       |        | BP [8] |       |        |
|-----------------|----------|----------|----------------|----------------|-----------------|-------|--------|--------|-------|--------|
|                 | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | $LB$            | $UB$  | $time$ | $LB$   | $UB$  | $time$ |
| r01             | 144      | 1,280    | 9,016          | 796,427        | 6,724           | 6,724 | 17.6   | 6,724  | 6,724 | 171    |
| r02             | 142      | 1,246    | 8,765          | 763,268        | 6,771           | 6,771 | 11.3   | 6,771  | 6,771 | 200    |
| r03             | 139      | 1,188    | 8,403          | 716,330        | 6,473           | 6,473 | 10.4   | 6,473  | 6,473 | 250    |
| r04             | 151      | 1,406    | 9,919          | 919,467        | 6,342           | 6,342 | 11.4   | 6,342  | 6,342 | 565    |
| r05             | 142      | 1,266    | 8,745          | 760,628        | 6,408           | 6,408 | 12.3   | 6,408  | 6,408 | 458    |
| r06             | 148      | 1,381    | 9,497          | 861,204        | 7,550           | 7,550 | 10.3   | 7,550  | 7,550 | 136    |
| r07             | 141      | 1,253    | 8,617          | 743,949        | 6,889           | 6,889 | 13.7   | 6,889  | 6,889 | 210    |
| r08             | 138      | 1,191    | 8,262          | 698,277        | 6,057           | 6,057 | 4.1    | 6,057  | 6,057 | 289    |
| r09             | 129      | 1,027    | 7,229          | 571,102        | 6,358           | 6,358 | 9.5    | 6,358  | 6,358 | 241    |
| r10             | 150      | 1,409    | 9,766          | 898,183        | 6,508           | 6,508 | 13.1   | 6,508  | 6,508 | 227    |
| r11             | 208      | 2,247    | 19,281         | 2,501,166      | 7,654           | 7,654 | 57.2   | 7,654  | 7,654 | 2,973  |
| r12             | 199      | 2,055    | 17,646         | 2,189,139      | 7,690           | 7,690 | 53.7   | 7,690  | 7,690 | 3,161  |
| r13             | 217      | 2,449    | 20,987         | 2,841,217      | 7,500           | 7,500 | 105.2  | 7,500  | -     | tl     |
| r14             | 214      | 2,387    | 20,404         | 2,723,361      | 8,254           | 8,254 | 65.3   | 8,254  | 8,254 | 1,420  |
| r15             | 198      | 2,055    | 17,448         | 2,152,308      | 8,021           | 8,021 | 25.1   | 8,021  | 8,021 | 1,157  |
| r16             | 188      | 1,861    | 15,717         | 1,839,320      | 7,755           | 7,755 | 26.7   | 7,755  | 7,755 | 712    |
| r17             | 213      | 2,392    | 20,186         | 2,679,713      | 7,979           | 7,979 | 79.3   | 7,979  | 7,979 | 2,257  |
| r18             | 200      | 2,079    | 17,821         | 2,221,870      | 7,232           | 7,232 | 58.2   | 7,232  | 7,232 | 2,502  |
| r19             | 185      | 1,803    | 15,217         | 1,751,992      | 6,826           | 6,826 | 32.7   | 6,826  | 6,826 | 1,349  |
| r20             | 217      | 2,447    | 20,989         | 2,841,629      | 8,023           | 8,023 | 104.1  | 8,023  | 8,023 | 993    |
| r21             | 281      | 3,554    | 35,786         | 6,339,798      | 9,284           | 9,284 | 390.0  | -      | -     | tl     |
| r22             | 285      | 3,684    | 36,786         | 6,607,860      | 8,887           | 8,887 | 302.6  | -      | -     | tl     |
| r23             | 288      | 3,732    | 37,596         | 6,827,707      | 9,136           | 9,136 | 375.3  | -      | -     | tl     |
| r24             | 269      | 3,284    | 32,762         | 5,552,048      | 8,464           | 8,464 | 201.7  | 8,464  | -     | tl     |
| r25             | 266      | 3,177    | 32,068         | 5,376,273      | 8,426           | 8,426 | 225.6  | -      | -     | tl     |
| r26             | 284      | 3,629    | 36,557         | 6,546,162      | 8,819           | 8,819 | 439.6  | -      | -     | tl     |
| r27             | 259      | 3,019    | 30,392         | 4,959,581      | 7,975           | 7,975 | 248.1  | 7,975  | -     | tl     |
| r28             | 288      | 3,765    | 37,563         | 6,818,692      | 9,407           | 9,407 | 222.7  | 9,407  | -     | tl     |
| r29             | 281      | 3,553    | 35,787         | 6,340,075      | 8,693           | 8,693 | 388.0  | 8,693  | -     | tl     |
| r30             | 301      | 4,122    | 41,028         | 7,785,349      | 9,816           | 9,816 | 346.9  | 9,816  | -     | tl     |

**Table 5:** Results for the Max-col,  $R$  instances.

$u, v \in V$ , but the constraint is active only if  $u, v$  are the centers of simplicial stars associated with two distinct colors (see proposition 1). Hence, there is a large number of constraints which are rarely activated. In this case, better results are obtained by defining (18) as *lazy constraints* in the CPLEX MIP solver, i.e., constraints that are not added to the formulation, but checked only when an incumbent (integer) solution is found (and eventually added to the formulation when violated).

The three recent papers [15], [16] and [14] report computational experiments on randomly generated graphs and the DIMACS benchmark instances. We consider the same random graphs used for testing the Branch-and-Bound algorithm in [16], which also summarizes results obtained by solving the compact formulation from [15], having from 60 to 120 vertices and densities  $\delta \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ , as reported in Table 6. Each row of the table reports average results over 30 instances for the size of the original and transformed graphs. The table also reports the number of instances solved to optimality (*opt*) within 2 hours of computing time by the approach  $MWSS(\hat{G})$ , and the average computing time (for solved instances only). In the table we also report the number of optimally solved instances and the average computing time for [15] and [16], obtained with a PC equipped with an Intel i5 CPU at 2.67GHz. We estimate a time ratio of 14/9 between this PC and our cluster computer, and accordingly, we scaled our time limit to 4600 seconds. Reported computing times from other papers are not scaled.

The approach we propose can solve all instances with 60 vertices, while 17 instances with 70

| $\delta$ | $G$      |          | $\hat{G}$      |                | $MWSS(\hat{G})$ |             | MIP [15]   |             | BB [16]    |             |
|----------|----------|----------|----------------|----------------|-----------------|-------------|------------|-------------|------------|-------------|
|          | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | <i>opt</i>      | <i>time</i> | <i>opt</i> | <i>time</i> | <i>opt</i> | <i>time</i> |
| 10       | 60       | 179.1    | 1,590.9        | 58,084.5       | 30              | 1.8         | 30         | 0.0         | 30         | 0.0         |
| 30       | 60       | 534.8    | 1,235.2        | 38,362.7       | 30              | 106.7       | 30         | 506.0       | 30         | 0.0         |
| 50       | 60       | 883.2    | 886.8          | 21,465.4       | 30              | 140.5       | 19         | 1,825.0     | 30         | 0.6         |
| 70       | 60       | 1,234.6  | 535.4          | 8,442.3        | 30              | 8.6         | 27         | 942.0       | 30         | 1.6         |
| 90       | 60       | 1,589.3  | 180.7          | 1,044.4        | 30              | 0.0         | 30         | 1.0         | 30         | 0.0         |
| 10       | 70       | 242.6    | 2,172.4        | 93,040.1       | 30              | 5.8         | 30         | 0.0         | 30         | 0.0         |
| 30       | 70       | 719.1    | 1,695.9        | 62,018.6       | 17              | 3,135.1     | 15         | 4,005.0     | 30         | 0.1         |
| 50       | 70       | 1,205.8  | 1,209.2        | 34,317.8       | 23              | 1,705.8     | 0          |             | 30         | 6.7         |
| 70       | 70       | 1,688.7  | 726.3          | 13,353.5       | 30              | 13.9        | 6          | 236.0       | 30         | 21.0        |
| 90       | 70       | 2,177.5  | 237.5          | 1,544.3        | 30              | 0.0         | 30         | 258.0       | 30         | 0.1         |
| 10       | 80       | 312.7    | 2,847.3        | 140,007.9      | 30              | 63.7        | 30         | 1.3         | 30         | 0.0         |
| 30       | 80       | 950.5    | 2,209.5        | 92,399.3       | 0               |             |            |             | 30         | 22.7        |
| 50       | 80       | 1,573.0  | 1,587.0        | 51,780.9       | 2               | 4,482.9     |            |             | 28         | 324.0       |
| 70       | 80       | 2,217.2  | 942.8          | 19,745.3       | 30              | 200.3       |            |             | 21         | 1,817.0     |
| 90       | 80       | 2,843.8  | 316.2          | 2,385.2        | 30              | 0.1         | 27         | 659.0       | 30         | 6.6         |
| 10       | 90       | 402.1    | 3,602.9        | 199,697.2      | 30              | 346.4       |            |             | 30         | 0.0         |
| 30       | 90       | 1,200.1  | 2,804.9        | 132,520.9      | 0               |             |            |             | 30         | 52.0        |
| 50       | 90       | 2,005.4  | 1,999.6        | 73,237.9       | 0               |             |            |             |            |             |
| 70       | 90       | 2,799.7  | 1,205.3        | 28,736.1       | 30              | 691.3       |            |             |            |             |
| 90       | 90       | 3,598.9  | 406.1          | 3,502.0        | 30              | 0.1         |            |             | 30         | 161.0       |
| 10       | 100      | 504.7    | 4,445.3        | 274,114.9      | 30              | 491.6       | 30         | 15.5        | 30         | 0.0         |
| 10       | 120      | 715.8    | 6,424.2        | 477,499.4      | 2               | 4,489.1     | 15         | 1,673.0     | 30         | 2.6         |

**Table 6:** Results for the ECP, random instances from [16].

vertices and  $\delta = 0.3$  and 23 instances with 70 vertices and  $\delta = 0.5$  are solved. When 80 or more vertices are considered, instances with  $\delta = 0.1, 0.7, 0.9$  remain tractable, while instances with  $\delta = 0.3, 0.5$  confirm to be difficult for the approach. Considering the same instances, complete results for the formulation in [15] are available up to 70 vertices (300 instances in total). The formulation in [15] can be solved to optimality for 217 instances, while the approach we propose can solve 280 of the same instances. The Branch-and-Bound algorithm [16] performs very well on these random problems, indeed, it can solve to optimality all the 300 instances with up to 70 vertices, and 141 instances with 80 vertices, while our approach can solve 92 instances with 80 vertices. Our approach seems to have a better performance on dense graphs: it is from one to four orders of magnitude faster for instances with  $\delta = 0.9$  and 70 or more vertices (both approaches solve all instances in these classes), and can solve all instances with 80 vertices and  $\delta = 0.7$ , while the algorithm in [16] can solve only 21 instances in this class.

Concerning the Branch-and-Cut algorithm in [14], it considers 5 instances per class, with up to 70 vertices. It can solve 2 instances with 60 vertices and  $\delta = 0.3$ , and 3 instances with 60 vertices and  $\delta = 0.5$ , while no instance with 70 vertices and  $\delta = 0.3$  or  $\delta = 0.5$  is solved within a time limit of 2 hours on a 1.8GHz PC with 1 GB RAM. This allows us to conclude that the method we propose has a better performance than [14] on randomly generated instances.

We also experimented our method on 54 DIMACS instances and 4 *kneser* graphs considered in [15], and restrict the set to 42 instances for which the auxiliary graph  $\hat{G}$  has at most 30,000 vertices. A subset of these instances was considered in [14] as well. Table 7 reports the size of the original and transformed graphs, and the results obtained by the presented method (columns  $MWSS(\hat{G})$ ), followed by the computing times from [15], [14] and [16], from the respective papers. The approach we propose performs satisfactorily on these instances, and solves to optimality

| <i>instance</i> | $G$      |          | $\hat{G}$      |                | $MWSS(\hat{G})$ |      |         | MIP [15] | BC [14] | BB [16] |
|-----------------|----------|----------|----------------|----------------|-----------------|------|---------|----------|---------|---------|
|                 | $ V(G) $ | $ E(G) $ | $ V(\hat{G}) $ | $ E(\hat{G}) $ | $LB$            | $UB$ | $time$  | $time$   | $time$  | $time$  |
| miles750        | 128      | 2,113    | 6,015          | 436,137        | 31              | 31   | 1.84    | 0.0      | 171.0   | tl*     |
| miles1000       | 128      | 3,216    | 4,912          | 323,889        | 42              | 42   | 1.07    | 0.0      | 267.0   | tl*     |
| miles1500       | 128      | 5,198    | 2,930          | 158,110        | 73              | 73   | 0.37    | 0.0      | 13.0    | 0.0     |
| zeroin.i.1      | 211      | 4,100    | 18,055         | 2,303,047      | 49              | 49   | 23.52   | 0.0      | 50.0    | 0.0     |
| zeroin.i.2      | 211      | 3,541    | 18,614         | 2,383,090      | 30              | 211  | tl      | 2.0      | 510.0   | tl      |
| zeroin.i.3      | 206      | 3,540    | 17,575         | 2,185,681      | 30              | 36   | tl      | 5.0      | 491.0   | tl      |
| queen6_6        | 36       | 290      | 340            | 5,092          | 7               | 7    | 0.82    | 1.0      | 1.0     | 0.0     |
| queen7_7        | 49       | 476      | 700            | 15,736         | 7               | 7    | 0.32    | 0.0      | 0.0     | 0.0     |
| queen8_8        | 64       | 728      | 1,288          | 40,344         | 9               | 9    | 3128.91 | 654.0    | 441.0   | 6.0     |
| queen8_12       | 96       | 1,368    | 3,192          | 162,036        | 12              | 12   | 46.62   | 5.0      |         | 3,079.0 |
| queen9_9        | 81       | 1,056    | 2,184          | 90,672         | 9               | 11   | tl      | tl       |         | 475.0   |
| queen10_10      | 100      | 1,470    | 3,480          | 184,600        | 10              | 12   | tl      | tl       |         | tl      |
| david           | 87       | 406      | 3,335          | 178,437        | 16              | 30   | tl      | 0.0      | 13.0    | 0.0     |
| 1-FullIns_3     | 30       | 100      | 335            | 5,342          | 4               | 4    | 0.08    | 0.0      | 2.0     | 0.0     |
| 2-FullIns_3     | 52       | 201      | 1,125          | 34,190         | 5               | 5    | 0.48    | 0.0      | 25.0    | 1.0     |
| 3-FullIns_3     | 80       | 346      | 2,814          | 137,405        | 6               | 6    | 1.55    | 0.0      | 85.0    | tl      |
| 4-FullIns_3     | 114      | 541      | 5,900          | 420,462        | 7               | 7    | 5.36    | 0.0      | 72.0    | tl      |
| 5-FullIns_3     | 154      | 792      | 10,989         | 1,073,628      | 8               | 8    | 22.48   | 0.0      | 268.0   | tl      |
| 1-FullIns_4     | 93       | 593      | 3,685          | 205,688        | 5               | 5    | 307.27  | 28.0     |         | 1,404.0 |
| mug88_1         | 88       | 146      | 3,682          | 206,951        | 4               | 4    | 12.80   | 1.0      |         | 109.0   |
| mug88_25        | 88       | 146      | 3,682          | 206,953        | 4               | 4    | 30.61   | 0.0      |         | 56.0    |
| mug100_1        | 100      | 166      | 4,784          | 307,175        | 4               | 4    | 125.01  | 1.0      |         | 4,425.0 |
| mug100_25       | 100      | 166      | 4,784          | 307,175        | 4               | 4    | 797.67  | 1.0      |         | 4,978.0 |
| mulsol.i.1      | 197      | 3,925    | 15,381         | 1,805,912      | 49              | 50   | tl      | 1.0      |         | 0.0     |
| mulsol.i.2      | 188      | 3,885    | 13,693         | 1,498,197      | 31              | 188  | tl      | tl       |         | tl      |
| 1-Insertions_4  | 67       | 232      | 1,979          | 80,730         | 4               | 5    | tl      | tl       |         | 1,055.0 |
| 2-Insertions_3  | 37       | 72       | 594            | 13,020         | 4               | 4    | 3.80    | 0.0      |         | 0.0     |
| 3-Insertions_3  | 56       | 110      | 1,430          | 49,500         | 4               | 4    | 425.56  | 8.0      |         | 1.0     |
| 4-Insertions_3  | 79       | 156      | 2,925          | 146,146        | 3               | 4    | tl      | 836.0    |         | 1,615.0 |
| DSJC125.1       | 125      | 736      | 7,014          | 545,250        | 5               | 6    | tl      | 214.0    |         | 0.0     |
| DSJC125.5       | 125      | 3,891    | 3,859          | 197,166        | 14              | 22   | tl      | tl       |         | tl      |
| DSJC125.9       | 125      | 6,961    | 789            | 9,503          | 44              | 44   | 0.42    | tl       |         | tl      |
| DSJC250.1       | 250      | 3,218    | 27,907         | 4,350,878      |                 | 250  | tl      | tl       |         | tl      |
| DSJC250.5       | 250      | 15,668   | 15,457         | 1,588,249      |                 | 250  | tl      | tl       |         | tl      |
| DSJC250.9       | 250      | 27,897   | 3,228          | 79,902         | 72              | 72   | 3437.65 | tl       |         | tl      |
| myciel4         | 23       | 71       | 182            | 2,051          | 5               | 5    | 0.29    | 0.0      | 5.0     | 0.0     |
| myciel5         | 47       | 236      | 845            | 21,810         | 6               | 6    | 171.40  | 149.0    |         | 0.0     |
| myciel6         | 95       | 755      | 3,710          | 206,615        | 5               | 7    | tl      | tl       |         | tl      |
| flat300_20_0    | 300      | 21,375   | 23,475         | 3,021,152      |                 | 300  | tl      | tl       |         | tl      |
| kneser7_2       | 21       | 105      | 105            | 770            | 6               | 6    | 0.25    | 0.0      | 6.0     | 0.0     |
| kneser7_3       | 35       | 70       | 525            | 10,780         | 3               | 3    | 0.28    | 0.0      | 2.0     | 0.0     |
| kneser9_4       | 126      | 315      | 7,560          | 611,940        | 3               | 3    | 150.08  | 0.0      | 809.0   | 0.0     |

**Table 7:** Results for the ECP, DIMACS instances.

27 out of 42 instances within 4600 seconds of computing time. It worth mention that we are able to solve to optimality, for the first time, instances DSJC125.9 instance DSJC250.9. The Branch-and-Bound algorithm [15] is the best performing approach for this set of instances, and can solve 31 instances to optimality within 2 hours of computing time on the slower computer mentioned above, hence, with comparable effort. The Branch-and-Bound algorithm [16] can solve 26 instances (the miles750 and miles1000 instances, where time limit is marked  $tl^*$ , can be solved by choosing a different strategy in the algorithm). Finally, the algorithm in [14] solves all the 19 considered instances (the other instances are not considered).

### 3.6 Computational results for the BPPC.

State-of-the-art algorithms for the BPPC, proposed by Fernandes-Muritiba et al. [19], Elhedhli et al. [20] and Sadykov and Vanderbeck [21] are all based on the Branch-and-Price framework. All these algorithms are tested on BPPC instances with up to 500 items (vertices), obtained by

adding random incompatibility graphs with densities in the range  $0.1 - 0.9$  to 80 BPP instances from Falkenauer [27], for a total of 720 instances. Mentioned Branch-and-Price algorithms can solve approximately 700 out of 720 instances. The most difficult ones are those with densities in the range  $0.3 - 0.4$ . The approach we propose, that is, formulating the MWSSP on graph  $\hat{G}$  with the additional capacity constraints (19), can solve almost all instances with density 0.9, and a few instances with density 0.8, within a comparable computing time. However, these instances are easily solved by Branch-and-Price algorithms. Thus, we conclude that, although the method we propose can solve BPPC instances, it is not competitive with state-of-the-art Branch-and-Price algorithms.

## 4 Conclusions

We exploited the idea of solving Coloring problems on a graph by solving Maximum Weight Stable Set problems on an auxiliary graph, the solutions of which are in one-to-one correspondence with the colorings of the original graph, and we discuss the relation that exists between the Stable Set of the auxiliary graph and the feasible solutions of the so-called Representative formulation by Campêlo, Corrêa and Campos [22]. The contribution of the paper is twofold: first, the extensive computational experiments we performed showed that we could solve some very hard instances of the Vertex Coloring Problem on dense graphs, while for the Max-coloring Problem, the current state-of-the-art algorithm is largely outperformed; second, we extended the idea to the Equitable Coloring Problem and to the Bin Packing Problem with Conflicts, and performed extensive computational experiments for these problems as well. The experiments showed that the proposed method performs well for the Equitable Coloring Problem, while it is not competitive with state-of-the-art algorithms for the Bin Packing Problem with Conflicts. Most of the best performing algorithms for Graph Coloring problems are based on Branch-and-Price or Branch-and-Cut frameworks, which are generally tough tasks to implement and tune. Our approach is instead based on the idea of generating an effective polynomial size Mixed-Integer Programming formulation and then solving it by using a general purpose solver as a black box, exploiting in this manner the effectiveness of modern Mixed-Integer Programming solvers. The continuous development and improvement of these solvers would automatically reflect on the performances of the presented method.

## 5 Acknowledgments

The authors would like to thank Isabel Méndez-Díaz and Daniel Severin for sharing the ECP instances. The authors are grateful to two anonymous referees for their careful reading and useful comments, that helped very much in improving the paper.

## References

- [1] D. Cornaz and V. Jost. A one-to-one correspondence between colorings and stable sets. *Oper Res Lett*, 36(6):673 – 676, 2008.

- [2] F.T. Leighton. A graph coloring algorithm for large scheduling problems. *J Res Nat Bur Stand*, 84(6):489–503, 1979.
- [3] D. de Werra. An introduction to timetabling. *Eur J Oper Res*, 19:151–162, 1985.
- [4] A. Gamst. Some lower bounds for a class of frequency assignment problems. *IEEE T Veh Technol*, 35(1):8–14, 1986.
- [5] F.C. Chow and J.L. Hennessy. The priority-based coloring approach to register allocation. *ACM T Prog Lang Sys*, 12(4):501–536, 1990.
- [6] T.K. Woo, S.Y.W. Su, and R. Newman Wolfe. Resource allocation in a dynamically partitionable bus network using a graph coloring algorithm. *IEEE T Commun*, 39(12):1794–1801, 2002.
- [7] E. Malaguti and P. Toth. A survey on vertex coloring problems. *Int T Oper Res*, 17:1–34, 2010.
- [8] E. Malaguti, M. Monaci, and P. Toth. An exact approach for the vertex coloring problem. *Discrete Optim*, 8:174–190, 2011.
- [9] S. Gualandi and F. Malucelli. Exact solution of graph coloring problems via constraint programming and column generation. *INFORMS J Comput*, 24(1):81–100, 2012.
- [10] S. Held, W. Cook, and E.C. Sewell. Maximum-weight stable sets and safe lower bounds for graph coloring. *Math Program Comput*, 4:363–381, 2012.
- [11] D. Brélaz. New methods to color the vertices of a graph. *Commun ACM*, 22:251–256, 1979.
- [12] P. San Segundo. A new dsatur-based algorithm for exact vertex coloring. *Computers & Operations Research*, 39(7):1724 – 1733, 2012.
- [13] I. Méndez-Díaz and P. Zabala. A branch-and-cut algorithm for graph coloring. *Discrete Appl Math*, 154(5):826–847, 2006.
- [14] L. Bahiense, Y. Frota, T. F. Noronha, and C. C. Ribeiro. A branch-and-cut algorithm for the equitable coloring problem using a formulation by representatives. *Discrete Appl Math*, 164:34–46, 2014.
- [15] I. Méndez-Díaz, G. Nasini, and D. Severín. A polyhedral approach for the equitable coloring problem. *Discrete Appl Math*, 164:413–426, 2014.
- [16] I. Méndez-Díaz, G. Nasini, and D. Severín. An exact dsatur-based algorithm for the equitable coloring problem. *arXiv preprint arXiv:1405.7011*, 2014.
- [17] E. Malaguti, M. Monaci, and P. Toth. Models and heuristic algorithms for a weighted vertex coloring problem. *J Heuristics*, 15:503–526, 2009.
- [18] F. Furini and E. Malaguti. Exact weighted vertex coloring via branch-and-price. *Discrete Optim*, 9(2):130–136, 2012.
- [19] A. E. Fernandes Muritiba, M. Iori, E. Malaguti, and P. Toth. Algorithms for the bin packing problem with conflicts. *INFORMS J Comput*, 22(3):401–415, 2010.
- [20] S. Elhedhli, L. Li, M. Gzara, and J. Naoum-Sawaya. A branch-and-price algorithm for the bin packing problem with conflicts. *INFORMS J Comput*, 23(3):404–415, 2011.

- [21] R. Sadykov and F. Vanderbeck. Bin packing with conflicts: A generic branch-and-price algorithm. *INFORMS J Comput*, 25(2):244–255, 2013.
- [22] M. Campêlo, V.A. Campos, and R.C. Corrêa. On the asymmetric representatives formulation for the vertex coloring problem. *Discrete Appl Math*, 156(7):1097–1111, 2008.
- [23] F. Bonomo, M. Giandomenico, and F. Rossi. A note on the cornaz–jost transformation to solve the graph coloring problem. *Information Processing Letters*, 113(18):649 – 652, 2013.
- [24] F. Furini, V. Gabrel, and I.-C. Ternier. Lower bounding techniques for dsatur-based branch and bound. *Electronic Notes in Discrete Mathematics*, 52:149 – 156, 2016.
- [25] D. Cornaz and P. Meurdesoif. Chromatic gallai identities operating on lovász number. *Mathematical Programming*, 144(1):347–368, 2014.
- [26] P. Ostergård. A fast algorithm for the maximum clique problem. *Discrete Appl Math*, 120:197 – 207, 2002.
- [27] E. Falkenauer. A hybrid grouping genetic algorithm. *J Heuristics*, 2:5–30, 1996.